Manim's Output Settings

This document will focus on understanding manim's output files and some of the main command-line flags available.

> **Note**
>
> This tutorial picks up where Quickstart left off, so please read that document before starting this one.

Manim output folders

At this point, you have just executed the following command.

```
manim -pql scene.py SquareToCircle
```

Let's dissect what just happened step by step. First, this command executes manim on the file `scene.py`, which contains our animation code. Further, this command tells manim exactly which `Scene` is to be rendered, in this case, it is `SquareToCircle`. This is necessary because a single scene file may contain more than one scene. Next, the flag *-p* tells manim to play the scene once it's rendered, and the *-ql* flag tells manim to render the scene in low quality.

After the video is rendered, you will see that manim has generated some new files and the project folder will look as follows.

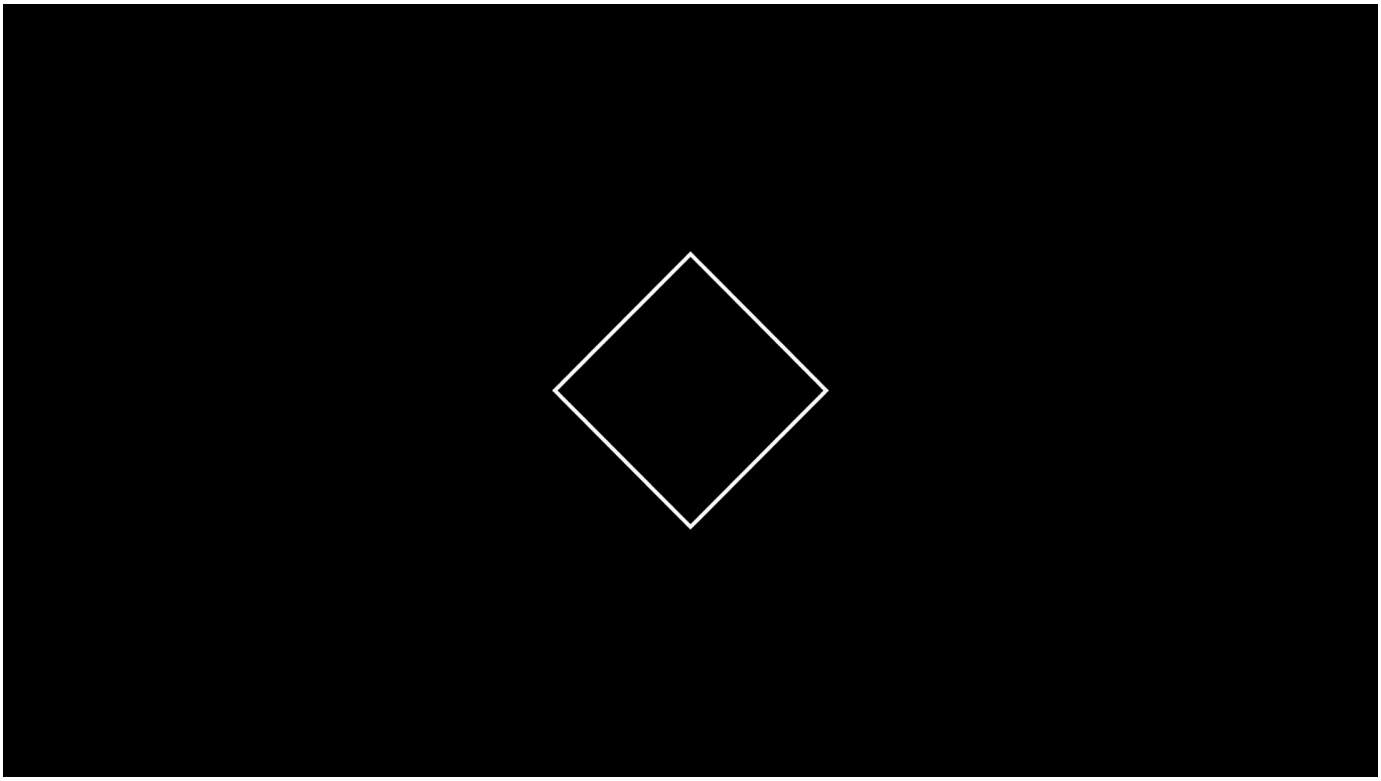
```
project/
├─scene.py
└─media
  ├─videos
  |  └─scene
  |     └─480p15
  |        ├─SquareToCircle.mp4
  |        └─partial_movie_files
  ├─text
  └─Tex
```

There are quite a few new files. The main output is in `media/videos/scene/480p15/SquareToCircle.mp4`. By default, the `media` folder will contain all of manim's output files. The `media/videos` subfolder contains the rendered videos. Inside of it, you will find one folder for each different video quality. In our case, since we used the `-l` flag, the video was generated at 480 resolution at 15 frames per second from the `scene.py` file. [t]put can be found inside `media/videos/scene/48` `ne/480p15/partial_movie_files` as well as `med` contain files that are used by manim internally.

You can see how manim makes use of the generated folder structure by executing the following command,

```
manim -pqh scene.py SquareToCircle
```

The `-ql` flag (for low quality) has been replaced by the `-qh` flag, for high quality. Manim will take considerably longer to render this file, and it will play it once it's done since we are using the `-p` flag. The output should look like this:



And the folder structure should look as follows.

```
project/
├─scene.py
└─media
  ├─videos
  | └─scene
  |   ├─480p15
  |   | ├─SquareToCircle.mp4
  |   | └─partial_movie_files
  |   └─1080p60
  |     ├─SquareToCircle.mp4
  |     └─partial_movie_files
  ├─text
  └─Tex
```


d a new folder `media/videos/1080p60`, which cc , 60 frames per second. Inside of it, you can fin `SquareToCircle.mp4`, as well as the corresponding `partial_movie_files`.

When working on a project with multiple scenes, and trying out multiple resolutions, the structure of the output directories will keep all your videos organized.

Further, manim has the option to output the last frame of a scene, when adding the flag `-s`. This is the fastest option to quickly get a preview of a scene. The corresponding folder structure looks like this:

```
project/
├─scene.py
└─media
  ├─images
  | └─scene
  |   ├─SquareToCircle.png
  ├─videos
  | └─scene
  |   ├─480p15
  |   | ├─SquareToCircle.mp4
  |   | └─partial_movie_files
  |   └─1080p60
  |     ├─SquareToCircle.mp4
  |     └─partial_movie_files
  ├─text
  └─Tex
```

Saving the last frame with `-s` can be combined with the flags for different resolutions, e.g. `-s -ql`, `-s -qh`

Sections

In addition to the movie output file one can use sections. Each section produces its own output video. The cuts between two sections can be set like this:

```python
def construct(self):
    # play the first animations...
    # you don't need a section in the very beginning as it gets created automatically
    self.next_section()
    # play more animations...
    self.next_section("this is an optional name that doesn't have to be unique")
    # play even more animations...
    self.next_section("this is a section without any animations, it will be removed")
```

All the animations between two of these cuts get concatenated into a single output video file. Be aware that you need at least one animation in each section. For example this wouldn't create an output video:

```
def construct(self):
    self.next_section()
    # this section doesn't have any animations and will be removed
    # but no error will be thrown
    # feel free to tend your flock of empty sections if you so desire
    self.add(Circle())
    self.next_section()
```

One way of fixing this is to wait a little:

```
def construct(self):
    self.next_section()
    self.add(Circle())
    # now we wait 1sec and have an animation to satisfy the section
    self.wait()
    self.next_section()
```

For videos to be created for each section you have to add the `--save_sections` flag to the Manim call like this:

```
manim --save_sections scene.py
```

If you do this, the `media` folder will look like this:

```
media
├── images
│   └── simple_scenes
└── videos
    └── simple_scenes
        └── 480p15
            ├── ElaborateSceneWithSections.mp4
            ├── partial_movie_files
            │   └── ElaborateSceneWithSections
            │       ├── 2201830969_104169243_1331664314.mp4
            │       ├── 2201830969_398514950_125983425.mp4
            │       ├── 2201830969_398514950_3447021159.mp4
            │       ├── 2201830969_398514950_4144009089.mp4
            │       ├── 2201830969_4218360830_1789939690.mp4
            │       ├── 3163782288_524160878_1793580042.mp4
            │       └── partial_movie_file_list.txt
            └── sections
                ├── ElaborateSceneWithSections_0000.mp4
                ├── ElaborateSceneWithSections_0001.mp4
                ├── ElaborateSceneWithSections_0002.mp4
                └── ElaborateSceneWithSections.json
```


ach section receives their own output video in th

JSON file in here contains some useful information for each section:

```
[
    {
        "name": "create square",
        "type": "default.normal",
        "video": "ElaborateSceneWithSections_0000.mp4",
        "codec_name": "h264",
        "width": 854,
        "height": 480,
        "avg_frame_rate": "15/1",
        "duration": "2.000000",
        "nb_frames": "30"
    },
    {
        "name": "transform to circle",
        "type": "default.normal",
        "video": "ElaborateSceneWithSections_0001.mp4",
        "codec_name": "h264",
        "width": 854,
        "height": 480,
        "avg_frame_rate": "15/1",
        "duration": "2.000000",
        "nb_frames": "30"
    },
    {
        "name": "fade out",
        "type": "default.normal",
        "video": "ElaborateSceneWithSections_0002.mp4",
        "codec_name": "h264",
        "width": 854,
        "height": 480,
        "avg_frame_rate": "15/1",
        "duration": "2.000000",
        "nb_frames": "30"
    }
]
```


This data can be used by third party applications, like a presentation system or automated video editing tool.

You can also skip rendering all animations belonging to a section like this:

```
def construct(self):
    self.next_section(skip_animations=True)
    # play some animations that shall be skipped...
    self.next_section()
    # play some animations that won't get skipped...
```


mmand line flags

When executing the command

```
manim -pql scene.py SquareToCircle
```

it specifies the scene to render. This is not necessary now. When a single file contains only one `Scene` class, it will just render the `Scene` class. When a single file contains more than one `Scene` class, manim will let you choose a `Scene` class. If your file contains multiple `Scene` classes, and you want to render them all, you can use the `-a` flag.

As discussed previously, the `-ql` specifies low render quality (854x480 15FPS). This does not look very good, but is very useful for rapid prototyping and testing. The other options that specify render quality are `-qm`, `-qh`, `-qp` and `-qk` for medium (1280x720 30FPS), high (1920x1080 60FPS), 2k (2560x1440 60FPS) and 4k quality (3840x2160 60FPS), respectively.

The `-p` flag plays the animation once it is rendered. If you want to open the file browser at the location of the animation instead of playing it, you can use the `-f` flag. You can also omit these two flags.

Finally, by default manim will output .mp4 files. If you want your animations in .gif format instead, use the `--format gif` flag. The output files will be in the same folder as the .mp4 files, and with the same name, but a different file extension.

This was a quick review of some of the most frequent command-line flags. For a thorough review of all flags available, see the thematic guide on Manim's configuration system.